

Entwicklung eines Powerpoint zu L^AT_EX-Beamer-Konverters auf Basis von Large Language Models

Ivan Kurilin

Matrikelnummer: 11158610

Modul: Cyber-Physische Systeme

Prüfer: Prof. Dr. Daniel Gaida

25. März 2026

Zusammenfassung

Dieser Bericht dokumentiert die Entwicklung eines Tools zur Konvertierung von PowerPoint-Präsentationen in $\text{\LaTeX}$ -Beamer-Code. Durch die Kombination von *Docling* für die Inhaltsanalyse und lokalen *Large Language Models* wird eine Pipeline realisiert, die Texte, Medien und Layout-Informationen extrahiert und in kompilierbaren $\text{\LaTeX}$ -Code überführt.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Theoretische Grundlagen von pptx Datei	1
1.2	Struktur von OpenXML-Präsentationen	2
1.3	Herausforderungen der Dokumentenanalyse	4
1.4	Large Language Models in der Codegenerierung	5
2	Analyse genutzter Technologien	5
2.1	Evaluation der Extraktionsverfahren	6
2.2	Inferenz-Infrastruktur	8
2.2.1	Modellwahl	8
2.3	Unabhängigkeit von Cloud-Modellen	10
3	Systementwurf und Architektur	11
3.1	Pipeline-Konzept und Modulbauweise	11
3.2	Extraktions-Layer	12
3.2.1	Strukturanalyse	12
3.2.2	Medien-Extraktion	13
3.3	Datenminimierung	14
3.3.1	Koordinatentransformation	14
3.3.2	Semantische Anreicherung	15
3.4	Agenten-basierte Synthese	18
3.4.1	Prompt-Engineering und Methodik	19
3.5	Syntax-Validierung	20
3.6	Kompilierung	21
4	Ergebnisse	21
4.1	Artefakte und Limitationen	21
4.1.1	Geometrische Inkonsistenzen bei Verschachtelungen	22
4.2	Vergleich mit bestehenden Lösungen	26
5	Fazit	29
5.1	Zusammenfassung der Erkenntnisse	29
5.1.1	Reflexion des KI-Einsatzes	29
5.2	Dezentrale Ansätze	30
5.3	Anwendungspotenzial	30
5.4	Abschließende Betrachtung	31

Literaturverzeichnis

1 Einleitung

In vielen professionellen oder akademischen Umgebungen sind Präsentationen das primäre Medium zur Vermittlung verschiedener Konzepte. Die Software zur Erstellung dieser Präsentationen verfügt über einen hohen Funktionsumfang und eine gewöhnliche Nutzerschaft. Während Microsoft PowerPoint aufgrund der grafischen Benutzeroberfläche und der Geschwindigkeit beim Layouting dominiert, bleibt LaTeX Beamer der bevorzugte Standard für den Einsatz im MINT Bereich. [The LaTeX Project, 2026; OTH Amberg-Weiden, 2026]

Oft werden erste Entwürfe schnell in PowerPoint erstellt. Wenn diese Inhalte später in ein LaTeX Dokument überführt werden müssen, gibt es bisher nur wenige Lösungen. Die manuelle Neuerstellung der Folien ist ein repetitiver und zeitintensiver Prozess. Entwickler müssen Texte kopieren, Tabellenumgebungen händisch programmieren und Bilder positionieren. Diese Arbeit bietet keinen kreativen Mehrwert und hält von den eigentlichen technischen Aufgaben ab.

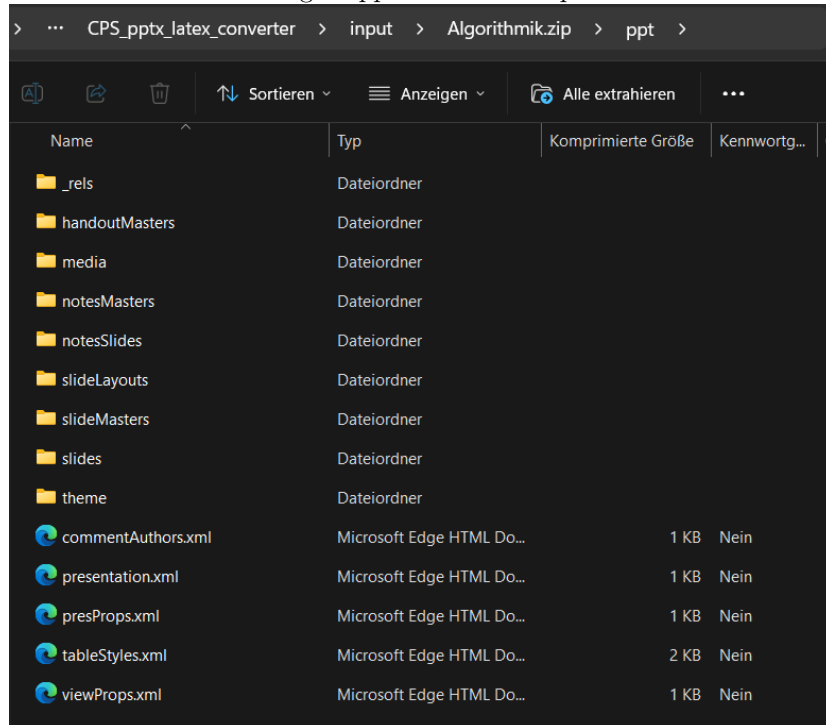
KI-Automatisierungen scheitern oft an komplexen Quelldateien. Beim einfachen Einlesen geht die visuelle Struktur der Folien verloren: Die Modelle erfassen zwar den Text, erkennen aber keine logischen Zusammenhänge. Listen werden dadurch zu bloßem Fließtext und Tabellenstrukturen lösen sich vollständig auf. Da der erzeugte Code eine intensive Nachbearbeitung erfordert, schwindet der Zeitvorteil der Automatisierung. Ein detaillierter Vergleich der verschiedenen Extraktionstools findet sich in Kapitel 2. Ein ähnliches Problem thematisieren Xu und andere Autoren in ihrer Studie zum Layout-Verständnis [Xu et al., 2020].

Ziel dieses Projekts ist die Entwicklung einer Pipeline, die diesen Prozess durch den Einsatz von künstlicher Intelligenz automatisiert. Durch die Kombination von Layout Analysis und Sprachmodellen soll die visuelle Struktur einer PowerPoint Folie erkannt und direkt in validen LaTeX Code übersetzt werden. Der Fokus liegt dabei auf der korrekten Rekonstruktion von Tabellen, Listen und der absoluten Positionierung von Elementen. Das Tool soll die Geschwindigkeit der grafischen Erstellung mit der typografischen Variabilität von LaTeX vereinen und den manuellen Aufwand auf ein Minimum reduzieren.

1.1 Theoretische Grundlagen von pptx Datei

Eine .pptx Datei ist weit mehr als nur das visuelle Dokument, das während einer Präsentation auf dem Bildschirm erscheint. Seit der Einführung des Office Open XML Standards durch Microsoft im Jahr 2007 handelt es sich bei diesem Dateiformat technisch gesehen um einen Container [Microsoft Corporation, 2007]. Die Datei ist ein komprimiertes Archiv, das mit einem gewöhnlichen Tool zur Datenkompression entpackt werden kann. Nach dem Entpacken wird die gesamte interne Struktur der Präsentation sichtbar, die aus einer hierarchischen Anordnung von Verzeichnissen und Dateien besteht.

Abbildung 1: .pptx-Datei als .zip-Archiv



Innerhalb dieser Archivstruktur werden die Informationen nicht als ein einziger Datenblock gespeichert, sondern liegen als separate Einheiten vor. Der Kern der Präsentation befindet sich im Verzeichnis für die Folien, in dem jede einzelne Folie als eigene XML Datei gespeichert ist. Diese XML Dateien enthalten die gesamte Logik über die Anordnung von Texten sowie die Definition von grafischen Formen.

Zusätzlich zur Struktur der Folien enthält das Archiv einen speziellen Ordner für Medien. Hier werden sämtliche Assets wie Bilder im Format .png oder Videos im Format .mp4 als eigenständige Dateien abgelegt. Diese Dateien liegen dort in ihrer ursprünglichen Form vor und sind über Referenzen in den XML Dokumenten mit den jeweiligen Folien verknüpft. Diese modulare Bauweise ist die technische Voraussetzung dafür, dass Bilder und andere Medienformate über Powerpoint gesteuert werden können. Die Metadaten über die Dateitypen und die Beziehungen zwischen den Objekten sind in speziellen Beziehungsdateien hinterlegt, die das Mapping innerhalb der Präsentation steuern [ISO/IEC, 2016].

1.2 Struktur von OpenXML-Präsentationen

Die Architektur von OpenXML basiert auf dem Prinzip der Trennung von Inhalten und deren Beziehungen [ISO/IEC, 2016]. Während die eigentlichen Daten in XML Dateien gespeichert werden, steuern separate Beziehungsdateien mit der Endung .rels die Verknüpfungen zwischen den einzelnen Komponenten. Für die Entwicklung einer Pipeline ist diese Abstraktionsebene entscheidend, da ein

Objekt auf einer Folie nicht direkt eingebettet ist, sondern über eine eindeutige ID referenziert wird.

Innerhalb der Folien XML Dokumente werden grafische Elemente über die Sprache DrawingML [DML, 2023] definiert. Jedes Element wie ein Textfeld oder eine geometrische Form wird als Shape Objekt deklariert [WordprocessingML, 2026]. Diese Objekte enthalten Informationen über ihre Geometrie, die Position auf dem Koordinatensystem der Folie sowie die visuellen Eigenschaften. Ein besonderes Merkmal ist die hierarchische Struktur der Textkörper innerhalb dieser Shapes. Texte werden in Absätze und Textläufe unterteilt, was eine präzise Extraktion der Formatierungen ermöglicht. Für den Konverter bedeutet dies, dass die logische Baumstruktur des XML Dokuments traversiert werden muss, um die Inhalte in der richtigen Reihenfolge zu erfassen. Die Abbildungen 3 und 4 veranschaulichen diese Struktur exemplarisch anhand von Folie 2.

Ein weiterer Bestandteil sind die Master-Layouts. Diese befinden sich physisch im Verzeichnis `ppt/slideMasters` und definieren die globalen Platzhalter sowie die Standardformatierungen für eine Gruppe von Folien. Die Pipeline muss in der Lage sein, diese Vererbungslogik zu interpretieren, um statische Elemente von dynamischen Inhalten zu unterscheiden (siehe Abbildung 2). Dadurch lassen sich festgelegte Kopf- und Fußzeilen identifizieren, sofern diese über den Master definiert wurden. Sind sie dort nicht vorhanden, liegen sie als reguläre Textelemente direkt in der jeweiligen Foliendatei vor.

Abbildung 2: Slide Masters

```
▼<p:sp>
  ▼<p:nvSpPr>
    <p:cNvPr id="1026" name="Titelplatzhalter 1"/>
    ▼<p:cNvSpPr>
      <a:spLocks noGrp="1"/>
    </p:cNvSpPr>
    ▼<p:nvPr>
      <p:ph type="title"/>
    </p:nvPr>
  </p:nvSpPr>
  ▼<p:spPr bwMode="auto">
    ▼<a:xfrm>
      <a:off x="1204913" y="520700"/>
      <a:ext cx="10799762" cy="900113"/>
    </a:xfrm>
```

Abbildung 3: Repräsentation des Bildobjekts slide2.xml in der XML-Baumstruktur

```

<p:pic>
  <p:nvPicPr>
    <p:cNvPr id="763" name="Google Shape;763;g118b4eb0048_0_551" descr="A screenshot of a cell phone
    Description automatically generated"/>
    <p:cNvPicPr preferRelativeResize="0"/>
    <p:nvPicPr>
      <p:blipFill rotWithShape="1">
        <a:blip r:embed="rId3">
          <a:alphaModFix/>
        </a:blip>
        <a:srcRect l="2998" r="1216"/>
        <a:stretch/>
      </p:blipFill>
      <p:spPr>
        <a:xfrm>
          <a:off x="6757172" y="2490907"/>
          <a:ext cx="5257027" cy="3392535"/>
        </a:xfrm>
        <a:prstGeom prst="rect">
          <a:avLst/>
        </a:prstGeom>
        <a:noFill/>
      </p:spPr>
    </p:pic>

```

Abbildung 4: Referenzierung auf das Bild in der .rels-Datei

```

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId3" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/image"
  Target="..\media/image2.png"/>
  <Relationship Id="rId2" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/notesSlide"
  Target="..\notesSlides/notesSlide2.xml"/>
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slideLayout"
  Target="..\slideLayouts/slideLayout2.xml"/>
  <Relationship Id="rId4" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/hyperlink"
  Target="https://www.bigocheatsheet.com/" TargetMode="External"/>
</Relationships>

```

1.3 Herausforderungen der Dokumentenanalyse

Die Vorverarbeitung von unstrukturierten Daten für ein Sprachmodell ist die Engstelle der Entwicklungspipeline. Die Datenstruktur ist hierbei die essenzielle Grundlage für die nachfolgende Verarbeitung durch das Large Language Model. Ohne eine geeignete Vorbereitung der Daten und deren strukturierte Nutzung anhand klar definierter Regeln ist es kaum möglich, eine konstante und deterministische Antwort vom Modell zu erhalten. Ein ungeordneter Datenstrom ohne präzise Segmentierung der Layout-Elemente führt dazu, dass das Modell den logischen Kontext verliert [Xu et al., 2020]. In der Folge verschlechtert sich die Qualität der generierten LaTeX-Codes.

Auf dem Markt existieren verschiedene Frameworks, die sich auf das Parsing von Dokumenten spezialisiert haben. Ein weit verbreitetes Tool in der Open Source Community ist Unstructured.io [Unstructured Technologies, 2026], das besonders häufig für die Entwicklung von RAG-Systemen eingesetzt wird. Daneben bietet IBM Research mit dem Framework Docling [Docling, 2026] eine Lösung an, die mittels Deep Learning Modellen spezifisch auf die Erkennung von Tabellen und Koordinaten ausgerichtet ist.

Die Auswahl des passenden Parsers ist entscheidend für die Stabilität des gesamten Workflows. Jede dieser Bibliotheken verfolgt unterschiedliche Ansätze bei der semantischen Klassifizierung von Objekten. Während einige Frameworks auf

starren Heuristiken basieren, nutzen moderne Lösungen neuronale Netze für die Layout-Erkennung. Die Entscheidung für ein Tool beeinflusst maßgeblich, wie präzise die Informationen über die absolute Positionierung der Elemente für die anschließende Generierung erhalten bleiben. Eine detaillierte Gegenüberstellung der Leistungsmerkmale und eine Bewertung für den spezifischen Einsatz in der Konvertierung von Präsentationsmedien findet sich im Kapitel 2 dieser Arbeit.

1.4 Large Language Models in der Codegenerierung

Nachdem die Dokument-Analyse eine strukturierte Basis geschaffen hat, fungieren Large Language Models als generative Instanz zur Übersetzung dieser Informationen in valide Information. Im Gegensatz zu klassischen, regelbasierten Transpilern, die auf vordefinierte Mapping-Logiken angewiesen sind und bei unbekannten Verschachtelungen an Grenzen stoßen, können LLMs diese fehlenden Logiken kompensieren [Agater, 2011]. Durch In-Context Learning interpretieren sie semantische Zusammenhänge direkt aus dem Prompt und überführen diese flexibel in die Ziel-Syntax von LaTeX [Shah, 2026; Qingxiu Dong, 2022].

Die zentrale Stärke dieser Modelle liegt also in der Fähigkeit zum In-Context Learning. Das bedeutet, dass das Modell eine Aufgabe allein durch die im Prompt bereitgestellten Informationen lösen kann, ohne explizit für die Struktur von PowerPoint-Dateien trainiert werden zu müssen. Stattdessen werden die extrahierten Metadaten und Koordinaten innerhalb des Prompts so aufbereitet, dass das Modell die räumliche Anordnung der Elemente versteht und in entsprechende LaTeX-Umgebungen übersetzt. Ein kritischer Aspekt in diesem Prozess ist die Syntax-Stabilität. Da die Ausgabe von Sprachmodellen stochastisch erfolgt, ist die Implementierung von Mechanismen zur Validierung des generierten Codes notwendig, um Kompilierungsfehler in der LaTeX-Umgebung zu vermeiden.

In dieser Pipeline übernehmen LLMs somit die Rolle eines intelligenten Transpilers, der die Brücke zwischen der rein geometrischen Beschreibung einer Folie und der logischen Strukturierung eines wissenschaftlichen Dokuments schlägt. Die Zuverlässigkeit dieses Schritts hängt maßgeblich von der gezielten Aufbereitung der Eingabedaten im Prompt ab. Eine strukturierte Darstellung der Metadaten stellt sicher, dass das Kontextfenster optimal genutzt und Halluzinationen bei der Positionierung der Elemente minimiert werden.

2 Analyse genutzter Technologien

Die Auswahl einer geeigneten Extraktions-Engine ist eine wichtige Design-Entscheidung für Stabilität und Genauigkeit der Konvertierungspipeline. In diesem Kapitel werden die technologischen Ansätze untersucht, die für die Überführung von unstrukturierten Präsentationsdaten in ein maschinenlesbares Format infrage kommen. Da die Qualität des generierten LaTeX-Codes direkt von der Integrität der Eingangsdaten abhängt, liegt der Fokus des Vergleichs auf der präzisen Erkennung von Layout-Elementen, deren hierarchischen Relationen und der direkten wiederverwendung für das LLM.

Hierbei werden insbesondere die Frameworks Unstructured und Docling gegenübergestellt. Laut GitHub-Statistiken und Nutzerbewertungen auf Plattfor-

men wie Reddit bekommt insbesondere Docling sehr positive Rückmeldungen von Entwicklern [Martineau, 2026]. Durch eine Analyse ihrer Funktionsweisen und Einschränkungen wird die technologische Basis für den anschließenden Systementwurf definiert. Ziel ist es, das Framework zu identifizieren, das die höchste Konsistenz bei der Extraktion von Tabellen, Listen und absoluten Positionen bietet, um den Determinismus des nachgelagerten Sprachmodells zu gewährleisten.

2.1 Evaluation der Extraktionsverfahren

Tabelle 1: SWOT-Analyse der Extraktions-Technologien

Kategorie	Docling	Unstructured.io
Stärken	Präzise Erkennung von Tabellenstrukturen und geometrischen Anordnungen [Auer et al., 2024].	Hohe Bekanntheit und breite Unterstützung von über 25 Dateiformaten [Docs, 2026].
Schwächen	Fehlende Video-Extraktion; unterstützt primär Image-Embedding [DocDocs, 2024].	Kostenfreie Nutzung eingeschränkt; setzt Account und API-Key voraus [Pricing, 2026; API, 2026].
Chancen	Nahtlose Integration in lokale Pipelines; volle Datenkontrolle ohne Cloud-Abhängigkeit [Martineau, 2026].	Schnelle Pipeline-Erstellung durch über 30 fertige Cloud-Connectors [Connectors, 2025].
Risiken	Technologische Abhängigkeit von der Modellwartung durch IBM Research [Auer et al., 2024].	Langfristige Abhängigkeit von Preismodellen und potenziellen Lizenzänderungen [Pricing, 2026].

Wie die Analyse zeigt, bietet keines der betrachteten Tools eine vollständig deckungsgerechte Lösung für die Konvertierung von PowerPoint zu LaTeX. Ein massives Problem in der praktischen Anwendung ist die enorme Datenmenge: Bereits ein mittelkomplexes Dokument resultiert nach dem Parsing oft in mehreren tausend Zeilen JSON-Code pro Folie. Dieser Overhead überfüllt das Context Window des Sprachmodells, wodurch die Präzision der Antwort sinkt und das Risiko für Halluzinationen steigt.

Ein Beispiel für diesen Overhead ist die Tabellenausgabe bei Docling. Um zu erkennen, welcher Inhalt in die erste Zeile gehört, müssen unzählige Blöcke nach dem `start_row_offset_idx` gefiltert werden. Was für klassische Algorithmen ein struktureller Vorteil ist, wird für ein LLM zum Nachteil [Liu et al., 2023]: Die enorme Datenmenge bei gleichzeitig geringer Informationsdichte erschwert die präzise Verarbeitung und verschwendet wertvolle Tokens.

Listing 1: Beispiel für den Daten-Overhead bei Tabellenzellen

```
{
  "row_span": 1,
  "col_span": 1,
  "start_row_offset_idx": 0,
  "end_row_offset_idx": 1,
```

```

    "start_col_offset_idx": 0,
    "end_col_offset_idx": 1,
    "text": "Description",
    "column_header": true,
    "row_header": false,
    "row_section": false,
    "fillable": false
  },
  {
    "row_span": 1,
    "col_span": 1,
    "start_row_offset_idx": 0,
    "end_row_offset_idx": 1,
    "start_col_offset_idx": 1,
    "end_col_offset_idx": 2,
    "text": "O-Notation",
    "column_header": true,
    "row_header": false,
    "row_section": false,
    "fillable": false
  },
  {
    "row_span": 1,
    "col_span": 1,
    "start_row_offset_idx": 0,
    "end_row_offset_idx": 1,
    "start_col_offset_idx": 2,
    "end_col_offset_idx": 3,
    "text": "Runtime if you double n",
    "column_header": true,
    "row_header": false,
    "row_section": false,
    "fillable": false
  },
  {
    "row_span": 1,
    "col_span": 1,
    "start_row_offset_idx": 0,
    "end_row_offset_idx": 1,
    "start_col_offset_idx": 3,
    "end_col_offset_idx": 4,
    "text": "Example Algorithm",
    "column_header": true,
    "row_header": false,
    "row_section": false,
    "fillable": false
  }
}

```

Um eine konstante und deterministische Ausgabe zu garantieren, ist eine Minimierung der Datenmenge unumgänglich. Es ist notwendig, ein reines JSON-Objekt zu konstruieren, das ausschließlich die für die LaTeX-Generierung kritischen Attribute (wie Textinhalt, relative Position und Tabellenstruktur) filtert. In diesem Zusammenhang stellt sich die Frage, ob eine direkte Extraktion über die Bibliothek `python-pptx` [Canny, 2013] langfristig effizienter wäre.

Diese Bibliothek liest PowerPoint-Dateien direkt aus und liefert Inhalte wie Texte, Tabellen oder Bilder als einzelne Elemente zurück. Dadurch lässt sich selbst bestimmen, welche Informationen gesammelt werden, wodurch Daten-Ballast und die Abhängigkeit von externen Frameworks wie Docling vermieden werden. Im Rahmen dieser Arbeit wurde dennoch mit Docling gearbeitet, da es die Informationen bereits fertig aufbereitet zurückgibt und somit eine schnellere Umsetzung ermöglichte.

Eine detaillierte Darstellung der entwickelten JSON-Strukturen sowie die Validierung der daraus resultierenden Konvertierungsqualität erfolgt im Kapitel 3 und 4 bei den Datenminimierung und Ergebnissen.

2.2 Inferenz-Infrastruktur

Die lokale Bereitstellung des LLMs erfordert eine Inferenz-Infrastruktur, welche die Modellverwaltung sowie die Kommunikation mit der Pipeline übernimmt. Im Rahmen dieses Projekts wurde Ollama als primäres Framework gewählt. Die Entscheidung für Ollama basiert auf dem Vorteil der lokalen Datensicherheit:

Da die Verarbeitung auf eigener Hardware erfolgt, bleiben sensible Daten innerhalb der eigenen Infrastruktur. Zusätzlich entfallen nutzungsabhängige Token-Kosten kommerzieller Cloud-Anbieter. Das Framework ermöglicht ebenfalls den Zugriff auf verschiedene vorkonfigurierte Modelle sowie eine einfache Installation.

Durch die Bereitstellung einer REST-Schnittstelle lässt sich Ollama in die bestehende Python-Applikation integrieren. Die zugehörige Anwendung erlaubt es zudem, Prompts vorab interaktiv zu evaluieren. Da die Inferenz-Parameter in dieser Testumgebung identisch mit der API-Schnittstelle sind, bleibt die Konsistenz der Ergebnisse zwischen Testphase und produktiver Anwendung gewahrt. Als weitere technologische Möglichkeit auf dem Markt existiert zudem LM Studio [LM Studio Team, 2026]. Ein wesentlicher Unterschied liegt hier in der Kontrolle über die Hardware-Ressourcen. Im Gegensatz zu Ollama erlaubt LM Studio die explizite Zuweisung von Systemspeicher oder Grafikspeicher für das jeweilige Modell, was insbesondere bei spezifischen Hardware-Optimierungen von Vorteil sein kann. Für den Workflow dieser Arbeit deckte Ollama jedoch alle notwendigen Anforderungen an die Automatisierung und Stabilität vollständig ab.

2.2.1 Modellwahl

Die Auswahl des Sprachmodells bestimmt die Qualität des erzeugten LaTeX-Codes. Da die Inferenz lokal durchgeführt wird, ist die Modellwahl jedoch direkt an die Hardware-Ressourcen des Test-Systems gebunden. Aufgrund der verfügbaren Rechenleistung stieß das System bei Modellen mit einer Parametergröße von ca. 8 Milliarden (8B) an seine Leistungsgrenzen, weshalb der Fokus auf Modellen in dieser Größenordnung lag.

Um die Ergebnisse zu evaluieren, wurden die Modelle anhand der Konvertierung von vier PowerPoint-Folien in LaTeX-Code verglichen. Als Benchmark dienten dabei die syntaktische Korrektheit des Codes sowie die Geschwindigkeit:

- deepseek-r1:8b: Trotz der hohen Bekanntheit von DeepSeek im Web konnten diese Modelle für die Code-Generierung in diesem spezifischen Kontext

keine ausreichend konsistenten Ergebnisse liefern [olldeep, 2025].

- qwen3:8b: Dieses Modell lieferte qualitativ hochwertige Ergebnisse. Da es sich hierbei jedoch um ein Thinking -Modell handelt, lag die Verarbeitungszeit bei ca. zwei bis drei Minuten pro Folie. Diese Inferenzgeschwindigkeit erwies sich für den Betrieb einer zeiteffizienten automatisierten Pipeline als unzureichend [ollq3, 2025].
- qwen2.5-coder:7b: Dieses Modell stellte sich als Kompromiss. Es verkürzte die Verarbeitungszeit auf 20 bis 30 Sekunden pro Folie, lieferte jedoch eine geringere Code-Qualität als das 8B-Modell [ollq25, 2025].

Der direkte Vergleich der Geschwindigkeit sowie der erzeugten Syntax (siehe Abbildungen 5 und 6) verdeutlicht den Zeitgewinn durch das non-Thinking Coder-Modell. Der visuelle Abgleich der Ergebnisse (siehe Abbildungen 8 und 9) zeigt jedoch, dass qwen3:8b die Promptregeln präziser befolgt und dem Original stärker entspricht [qwen2.5-coder:7b, 2026; qwen3:8b, 2026].

Abbildung 5: Generierungsergebnis des qwen3:8b-Modells

```
-> Generating LaTeX for Slide 1 (1/4)...
2026-03-06 12:55:00,220 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 2 (2/4)...
2026-03-06 12:56:02,899 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 3 (3/4)...
2026-03-06 12:57:35,285 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 4 (4/4)...
2026-03-06 12:59:59,231 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
Assembling final document...
```

Abbildung 6: Generierungsergebnis des qwen2.5-coder:7b-Modells

```
-> Generating LaTeX for Slide 1 (1/4)...
2026-03-06 12:48:36,663 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 2 (2/4)...
2026-03-06 12:48:51,512 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 3 (3/4)...
2026-03-06 12:49:15,746 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
-> Generating LaTeX for Slide 4 (4/4)...
2026-03-06 12:49:41,020 - INFO - HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
Assembling final document...
```

Abbildung 7: Vergleich der Verarbeitungsgeschwindigkeit pro Folie

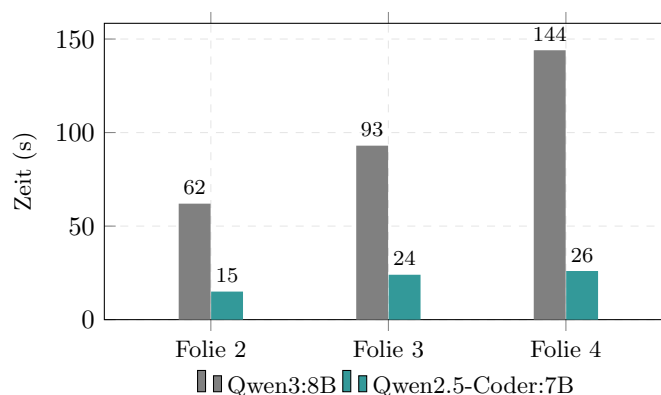


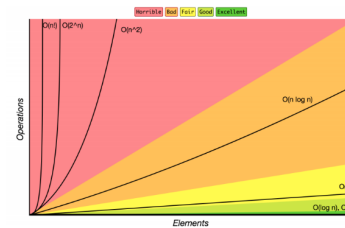
Abbildung 8: Visueller Vergleich der gerenderten Folie: qwen3:8b

Algorithmen - Laufzeitanalyse von Algorithmen durch Asymptotische Notation

Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Runtime if you double n	Example Algorithm
constant	$O(1)$	unchanged	Accessing an index of an array
logarithmic	$O(\log^2 n)$	increases slightly	Binary search
linear	$O(n)$	doubles	Looping over an array
log-linear	$O(n \log^2 n)$	slightly more than doubles	Merge sort algorithm
quadratic	$O(n^2)$	quadruples	Nested loops!
exponential	$O(2^n)$	multiplies drastically	Fibonacci with recursion



06.03.2026

Seite 2

Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

<https://www.khanacademy.com/>

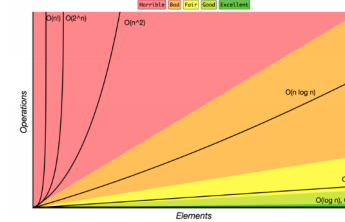
Abbildung 9: Visueller Vergleich der gerenderten Folie: qwen2.5-coder:7b

Algorithmen - Laufzeitanalyse von Algorithmen durch Asymptotische Notation

Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description	O-Notation	Example Algorithm
Runtime if you double n		
constant	$O(1)$	Accessing an index of an array
unchanged		
logarithmic	$O(\log^2 n)$	Binary search
increases slightly		
linear	$O(n)$	Looping over an array
doubles		
log-linear	$O(n \log^2 n)$	Merge sort algorithm
slightly more than doubles		
quadratic	$O(n^2)$	Nested loops!
quadruples		
...		
exponential	$O(2^n)$	



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

<https://www.khanacademy.com/>

2.3 Unabhängigkeit von Cloud-Modellen

Der Einsatz von Large Language Models erfolgt in der Praxis häufig über Cloud-Schnittstellen großer Technologieunternehmen wie OpenAI, Google oder Microsoft. Diese Abhängigkeit von externen Infrastrukturen bringt jedoch signifikante Risiken im Hinblick auf die Datensicherheit mit sich. In dieser Arbeit wird daher ein konsequent dezentraler Ansatz verfolgt, der durch die Nutzung von Ollama die Inferenz vollständig auf lokaler Hardware realisiert.

Ein wesentlicher Aspekt ist der Schutz geistigen Eigentums. Insbesondere im akademischen Kontext enthalten Präsentationen oft unveröffentlichte Forschungsergebnisse oder sensible Konzepte. Die Übermittlung dieser Daten an externe Server ist aus Datenschutzsicht kritisch zu bewerten. Durch die lokale Ausführung wird garantiert, dass keine Inhalts-Fragmente die eigene Infrastruktur verlassen. Darüber hinaus sichert dieser Ansatz die Unabhängigkeit von den Preismodellen und der Verfügbarkeit kommerzieller Anbieter.

3 Systementwurf und Architektur

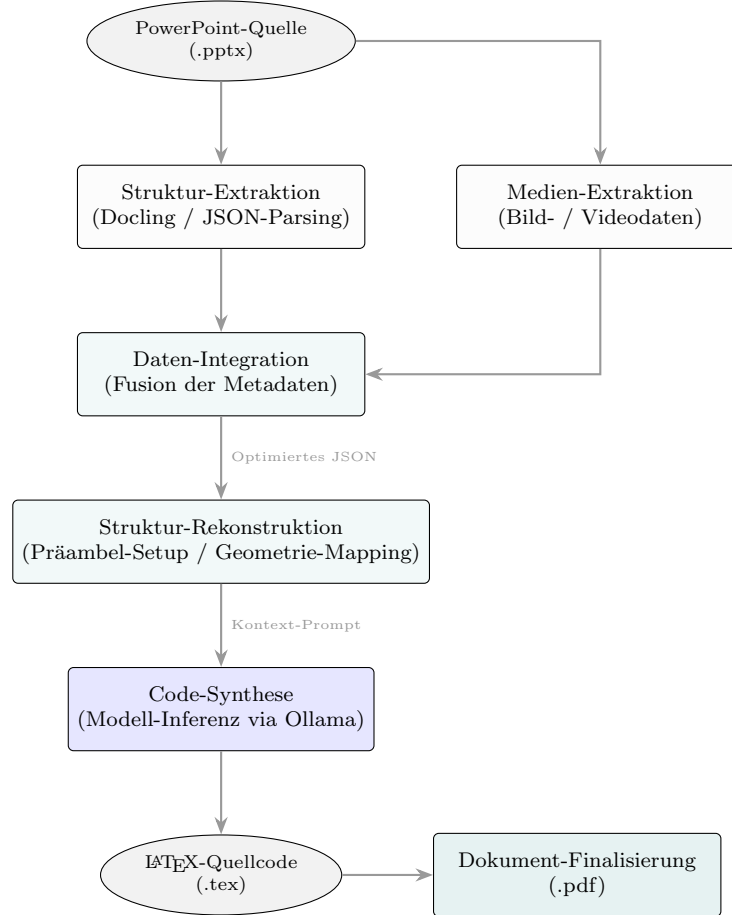
Dieses Kapitel behandelt die softwaretechnische Architektur des Konverters und die zugrunde liegenden Entwurfsentscheidungen. Das primäre Ziel des Systems ist die automatisierte Transformation von Präsentationsdaten in satzfähige LaTeX-Beamer-Dokumente. Der Entwurf basiert auf einer modularen Pipeline-Struktur, die eine strikte Trennung zwischen Datenextraktion, Prozessierung und Codegenerierung sicherstellt.

3.1 Pipeline-Konzept und Modulbauweise

Die Implementierung der Systemarchitektur erfolgt nach dem Pipe-and-Filter-Entwurfsmuster (siehe graf. Darstellung 10). Diese Wahl ermöglicht die Dekomposition des Transformationsprozesses in diskrete, austauschbare Funktionseinheiten. Der Datenfluss beginnt mit der Auflösung der OpenXML-Struktur der Quelldatei. Hierbei agieren die Medienextraktion und die Layout-Analyse als initiale Filter, welche die Rohdaten in ein internes Repräsentationsformat überführen.

Durch diese modulare Entkopplung wird die Robustheit des Gesamtsystems gegenüber heterogenen Eingangsdaten erhöht. Ein wesentlicher Vorteil dieser Architektur liegt in der Unabhängigkeit der Prompt-Konstruktion von der physischen Datenextraktion. Die Kommunikation zwischen den einzelnen Filtern erfolgt über standardisierte Schnittstellen, was die Konsistenz der Datenströme über die gesamte Prozesskette hinweg gewährleistet und die Grundlage für eine deterministische Generierung des Zielcodes bildet.

Abbildung 10: Systemarchitektur der Konvertierungs-Pipeline.



3.2 Extraktions-Layer

Der Extraktions-Layer fungiert als Schnittstelle zwischen dem OpenXML-Containerformat und der internen Datenverarbeitung der Pipeline. Die primäre Aufgabe besteht in der Aggregation semantischer Informationen und der Isolation eingebetteter Ressourcen. Um eine vollständige Erfassung aller Folienobjekte bei gleichzeitiger Optimierung des Datenvolumens sicherzustellen, implementiert das System eine hybride Extraktionsstrategie. Diese unterteilt sich in die Analyse der logischen Dokumentenstruktur und die physische Sicherung der Mediendaten.

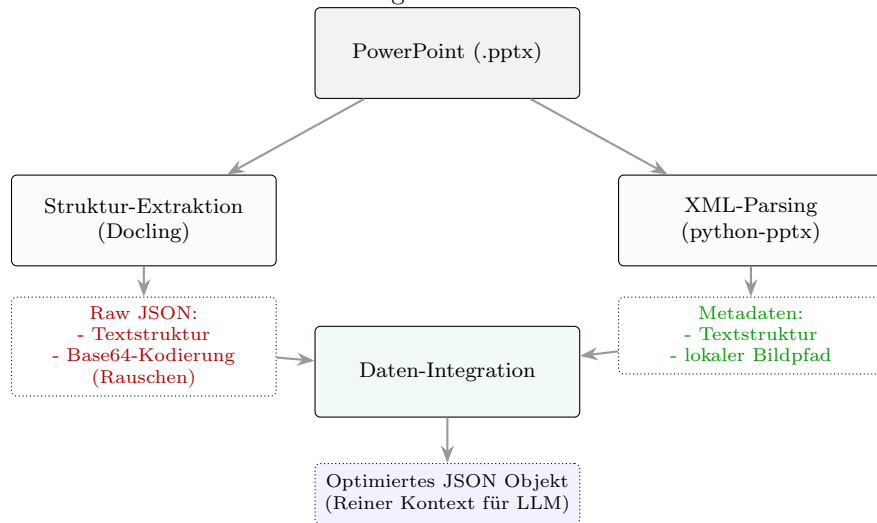
3.2.1 Strukturanalyse

Die strukturelle Analyse der Präsentationsdatei wird primär durch das Framework Docling realisiert. Dieses Modul übernimmt die Transformation der Folienhierarchie in ein JSON-Format, wobei Textelemente, Listenstrukturen und Tabellen klassifiziert werden.

Ein technisches Hindernis bei der Verwendung von Docling ist der Export grafischer Elemente als Base64-kodierte Zeichenfolgen. Da dies zu einer signifikanten

Vergrößerung des Datenstroms führt und die Performance bei der späteren Interaktion mit Sprachmodellen reduziert, findet eine Bereinigung statt. Es wird die Bibliothek `python-pptx` eingesetzt, um einen Zugriff auf die zugrunde liegenden XML-Knoten der Office Open XML-Spezifikation zu erhalten. Dies ermöglicht es, die durch Docling erzeugte Struktur mit Pfadreferenzen zu den grafischen Elementen anzureichen, während die Base64-Kodierungen eliminiert werden. Siehe Abbildung 11.

Abbildung 11: Daten-Fusion



3.2.2 Medien-Extraktion

Die Implementierung einer eigenständigen Medien-Extraktion adressiert funktionale Einschränkungen des Docling-Frameworks. Zwar sieht die Dokumentation für Markdown-Exporte die Referenzierung externer Bilder via `ImageRefMode.REFERENCED` vor, doch erbrachte diese Parameteränderung in der Praxis keine Vorteile. Zudem verbleiben visuelle Inhalte im hier benötigten JSON-Export, der über eine andere Methode (`export_to_dict()`) stattfindet, systembedingt als Base64-kodierte Zeichenfolgen [docling docs, 2026].

Da Docling zudem keine Unterstützung für Video-Objekte bietet, übernimmt die entwickelte Pipeline den Export aller Medienressourcen und ersetzt diese im Datenmodell durch Pfadreferenzen. Dies ermöglicht dem Modell eine Zuordnung der Assets ohne Prompt-Overhead.

Darüber hinaus erfordert die hierarchische Organisation des OpenXML-Formats eine spezifische Herangehensweise. Da Medienelemente innerhalb der Folienstruktur oft in tief verschachtelten Gruppen organisiert sind, führt eine lineare Verarbeitung zwangsläufig zu einem unvollständigen Datenmodell. Ein rekursiver Algorithmus ist daher die methodische Konsequenz, um die Integrität der extrahierten Ressourcen über alle Ebenen hinweg zu gewährleisten. Durch die anschließende Transformation der absoluten Positionsdaten in ein relatives Koordinatensystem wird die Abhängigkeit von exklusiven Maßeinheiten aufgelöst (siehe Abschnitt 3.3.1). Diese Normalisierung bildet die Grundlage für eine präzise Rekonstruktion der Layouts innerhalb der LaTeX-Beamer-Umgebung.

3.3 Datenminimierung

Die Transformation der Rohdaten in eine Intermediate Representation überführt unterschiedliche Datenstrukturen in ein einheitliches Schema. Durch das Mapping von semantischen Inhalten auf physische Medienreferenzen wird die Komplexität der Quelldaten für die nachfolgende Pipeline reduziert. Diese Konsolidierung verringert die Anzahl der Datenpunkte pro Folie signifikant, was die Performance bei der Verarbeitung durch LLMs steigert. Eine minimiertes Datenobjekt stellt sicher, dass das Context Window optimal genutzt wird und die Qualität der Codegenerierung durch ein besseres Signal-Rausch-Verhältnis steigt.

3.3.1 Koordinatentransformation

Um die Layout-Integrität beim Export von OpenXML nach LaTeX zu wahren, ist eine Normalisierung der Koordinatensysteme notwendig. PowerPoint nutzt intern English Metric Units, die für eine direkte Verwendung in LaTeX Beamer zu hochauflösend und einheitsabhängig sind. Die Transformation überführt die absoluten Bounding-Box-Werte in ein relatives Koordinatensystem im Intervall $[0.0, 1.0]$.

Dabei werden die Parameter für Position (x, y) und Ausdehnung (w, h) durch Division mit der gesamten Folienbreite bzw. Folienhöhe berechnet. Diese unit-agnostische Darstellung entkoppelt die Geometrie von festen Maßeinheiten wie pt oder cm und ermöglicht eine konsistente Skalierung innerhalb der Zielumgebung. Die Berechnung stellt zudem sicher, dass vertauschte Extremwerte innerhalb der Metadaten korrigiert werden (siehe Abbildung 12).

Abbildung 12: Python-Implementierung der Normalisierungslogik: Umrechnung absoluter EMUs in relative Koordinaten.

```
def calculate_geometry(bbox, page_width, page_height):
    """
    Berechnet relative LaTeX-Koordinaten (0.0-1.0).
    """
    if not bbox or page_width == 0 or page_height == 0:
        return None

    l = bbox.get('l', 0)
    t = bbox.get('t', 0)
    r = bbox.get('r', 0)
    b = bbox.get('b', 0)

    width_emu = abs(r - l)
    height_emu = abs(b - t)
    visual_top = min(t, b)

    rel_x = l / page_width
    rel_y = visual_top / page_height
    rel_w = width_emu / page_width
    rel_h = height_emu / page_height

    return {
        "x": round(max(0.0, min(1.0, rel_x)), 3),
        "y": round(max(0.0, min(1.0, rel_y)), 3),
        "w": round(max(0.0, min(1.0, rel_w)), 3),
        "h": round(max(0.0, min(1.0, rel_h)), 3)
    }
```

3.3.2 Semantische Anreicherung

Nach der geometrischen Aufbereitung folgt die Überführung der flachen Elementliste in ein strukturiertes Modell. Da das Ausgangsformat lediglich isolierte Objekte liefert, nutzt die Logik räumliche Heuristiken zur Erkennung von logischen Zusammenhängen. Elemente werden basierend auf ihren Koordinaten gruppiert, um Inhaltsbereiche zu identifizieren. Dieser Schritt ist essenziell, um die semantische Hierarchie der Folie wiederherzustellen und Fragmente in kohärente Elementgruppen zu überführen.

Um die Gruppierung zu verdeutlichen, zeigt Listing 2 einen Ausschnitt der flachen Ausgangsdaten. Der rohe Datensatz umfasst hier bereits 377 Zeilen, von denen die ersten betrachtet werden. Jedes erkannte Textelement liegt als separates Objekt in einem eindimensionalen Array vor. Die Koordinaten sind absolut in Pixeln angegeben und bilden auf dieser Ebene den einzigen Anhaltspunkt für einen inhaltlichen Zusammenhang.

Listing 2: Flache Elementliste (Ausschnitt vor der Transformation)

```
[
    {
        "self_ref": "#/texts/1",
        "parent": {
            "$ref": "#/groups/1"
        },
        "children": [],
        "content_layer": "body",
        "label": "list_item",
```

```

    "prov": [
    {
      "page_no": 1,
      "bbox": {
        "l": 1204913.0,
        "t": 5868000.0,
        "r": 12004675.0,
        "b": 1548000.0,
        "coord_origin": "BOTTOMLEFT"
      },
      "charspan": [
        0,
        489
      ]
    }
  ],
  "orig": "Abstrakte Datentypen",
  "text": "Abstrakte Datentypen",
  "enumerated": false,
  "marker": ""
},
{
  "self_ref": "#/texts/2",
  "parent": {
    "$ref": "#/groups/1"
  },
  "children": [],
  "content_layer": "body",
  "label": "list_item",
  "prov": [
  {
    "page_no": 1,
    "bbox": {
      "l": 1204913.0,
      "t": 5868000.0,
      "r": 12004675.0,
      "b": 1548000.0,
      "coord_origin": "BOTTOMLEFT"
    },
    "charspan": [
      0,
      489
    ]
  }
  ],
  "orig": "Liste, Stapel, Warteschlange, Tabelle (Dictionary)",
  "text": "Liste, Stapel, Warteschlange, Tabelle (Dictionary)",
  "enumerated": false,
  "marker": ""
},
// ... weitere 11 Listenelemente der page_no 1
]

```

Nach Durchlauf der räumlichen Heuristiken werden diese Fragmente aggregiert. Listing 3 demonstriert das Ergebnis: Die Einzelelemente sind nun in einem logischen Listen-Objekt zusammengefasst. Die Normalisierung der Koordinaten erfolgt durch `calculate.geometry()` (siehe Abbildung 12). Der Inhalt kann anschließend direkt an das LLM übergeben werden.

Listing 3: Strukturiertes Modell (Nach der Transformation)

```
{
  "type": "list",
  "geometry": {
    "x": 0.099,
    "y": 0.226,
    "w": 0.886,
    "h": 0.63
  },
  "items": [
    "Abstrakte Datentypen",
    "Liste, Stapel, Warteschlange, Tabelle
      (Dictionary)",
    "Datenstrukturen",
    "Array, Verkettete Liste"
    // ... restliche Eintraege
  ],
  "align": "t",
  "fontsize": "scriptsize"
}
```

Abbildungen 13 und 14 visualisieren diese Transformation am Beispiel eines Titel-Elements. Während der Rohdatensatz tief verschachtelte Metadaten enthält, reduziert die Intermediate Representation diese Komplexität deutlich. Die Klassifikation des Elements als Header erfolgt hier deterministisch durch die Auswertung der normalisierten y-Koordinate ($y < 0.03$). Die Bedingung besagt, dass die Oberkante des Elements in den obersten 3% der Folienhöhe liegt.

Abbildung 13: Vor der Transformation.

```
{
  "self_ref": "#/texts/12",
  "parent": {
    "$ref": "#/groups/0"
  },
  "children": [],
  "content_layer": "body",
  "label": "paragraph",
  "prov": [
    {
      "page_no": 1,
      "bbox": {
        "l": 1206000.0,
        "t": 365289.0,
        "r": 11991474.0,
        "b": 150124.0,
        "coord_origin": "BOTTOMLEFT"
      },
      "charspan": [
        0,
        74
      ]
    }
  ],
  "orig": "Algorithmik – Laufzeitanalyse von Algorithmen durch Asymptotische Notation",
  "text": "Algorithmik – Laufzeitanalyse von Algorithmen durch Asymptotische Notation"
},
```

Abbildung 14: Nach der Transformation.

```
{
  "type": "header",
  "geometry": {
    "x": 0.099,
    "y": 0.022,
    "w": 0.885,
    "h": 0.031
  },
  "text": "Algorithmik – Laufzeitanalyse von Algorithmen durch Asymptotische Notation",
  "fontsize": "3pt"
},
```

Nach der Identifikation als Header werden nur die essenziellen Daten in das neue Modell übernommen: der reine Textinhalt, die durch `calculate_geometry()` berechnete Position (siehe Abbildung 12) und Parameter zur Darstellung. Da der Header im fertigen Dokument besonders klein gesetzt ist, wird die entsprechende Schriftgröße (`fontsize`) direkt als Attribut im JSON notiert.

3.4 Agenten-basierte Synthese

Die Transformation des Datensatzes in LaTeX-Code ist der vorletzte Schritt der Pipeline. Um die Komplexität zu reduzieren, wird ein modularer Ansatz gewählt, der die Erstellung von Präambel, Folieninhalten und Postambel entkoppelt. Da statische Komponenten wie Paket-Importe oder die Titelfolie effizient als feste Präambel bzw. Postambel definiert werden können, wird die generative Last des Modells gezielt auf die Synthese des eigentlichen Folieninhalts beschränkt.

Die folgenden Unterabschnitte analysieren die Architektur des verwendeten Prompts, der sich aus drei Kernkomponenten zusammensetzt: dem syntaktischen Regel-

werk, den exemplarischen Anweisungen und den injizierten Nutzdaten aus dem JSON-Objekt.

3.4.1 Prompt-Engineering und Methodik

Die Code-Synthese basiert auf einer Konditionierung des Sprachmodells, um die Fehleranfälligkeit bei der Generierung von LaTeX-Strukturen zu reduzieren. Ein wesentliches Merkmal dieses Prozesses ist die Beschränkung des Kontextfensters auf einzelne Folieneinheiten. Durch diese Verarbeitung wird sichergestellt, dass das Modell während der Inferenz nicht durch globale Dokumentstrukturen abgelenkt wird, was das Risiko für Halluzinationen minimiert. Würde das Modell den gesamten Präsentationstext auf einmal erhalten, stiege die Gefahr, dass es Inhalte verschiedener Folien vermischt oder fälschlicherweise Layout-Entscheidungen fremder Seiten übernimmt. Die strikte seitenweise Verarbeitung zwingt das Modell stattdessen, sich ausschließlich auf die relevanten Daten der aktuell zu verarbeitenden Folie zu konzentrieren.

Der System-Prompt fungiert dabei als syntaktischer Guard. Wie in Listing 4 dargestellt, wird das Modell durch explizite Instruktionen auf ein vordefiniertes Zielschema fixiert. Ein zentraler Aspekt ist hierbei die Abbildung der normalisierten Geometriedaten ($x, y \in [0, 1]$) auf das `textpos`-Paket.

Dieser Ansatz wurde gewählt, um den Inhalt der ursprünglichen PowerPoint-Folie bestmöglich zu bewahren. Da PowerPoint-Elemente im Gegensatz zu Standard-LaTeX-Dokumenten nicht im Fließtext, sondern in absolut positionierten Containern vorliegen, bildet das `textpos`-Paket diese Logik technisch am präzisesten ab. Durch die Fixierung auf dieses Schema wird verhindert, dass das Sprachmodell versucht, Layout-Probleme durch unvorhersehbare LaTeX-Befehle zu lösen, was die Kompilierbarkeit des generierten Codes steigert. Die Verwendung von `minipage`-Umgebungen innerhalb der `textblock`-Container dient der technischen Abbildung der Objekthöhe (`h`) (siehe Listing 5). Sie ermöglicht eine konsistente vertikale Ausrichtung des Inhalts innerhalb der definierten Bounding Box und stellt sicher, dass der Text den vorgegebenen vertikalen Rahmen der ursprünglichen Folienelemente einhält.

Anstatt dem Modell gestalterische Freiheiten zu lassen, erzwingt der Prompt die Kapselung sämtlicher Inhalte in das definierte Schema:

Listing 4: Definition des Eingabedatensatzes

```
{
INPUT DATA:
You receive a JSON object representing a SINGLE slide with a
list of "elements".
Each element contains:
"type": text, list, codeblock, table, picture, video, ..
"geometry": { "x", "y", "w", "h" }
"content": "text", "items", "table_rows", "image_path",
"path" : ..

OUTPUT FORMAT RULES (STRICTLY FOLLOW):
1. **Frame Structure:**
- Start with '\begin{frame}[fragile]'. End with '\end{frame}'.
2. **Positioning (The Container):**
```

```

- For EACH element, generate a textblock: '\begin{textblock}
  {<w>}{<x>, <y>} ... \end{textblock}'.
3. **Content Layout (The Inner Box):**
- Inside EVERY textblock, wrap content in a minipage.
4. **Element-Specific Rendering:**
- "title", "header", "text": Output text.
Use '\textbf{...}' for titles.
}

```

Exemplarische Anweisung: Um die Präzision der semantischen Abbildung weiter zu erhöhen, kommen gezielte Input-Output-Beispiele zum Einsatz. Diese decken technische Grenzfälle ab, die über rein deklarative Regeln schwer zu erfassen sind. Beispielsweise lernt das Modell durch diese Paare die typografische Skalierung von Attributen in spezifische Befehle oder die Einbettung von Tabellenstrukturen in resizebox-Umgebungen zur Wahrung der strukturellen Integrität. Auch die differenzierte Setzung der Ausrichtungparameter ([t] vs. [b]) basierend auf dem semantischen Typ wird durch dieses demonstrative Vorgehen präzisiert (siehe Listing 5).

Listing 5: Beispielhafte Instanz für die In-Context-Konditionierung

```

Input 2 (List - [b]):
{
    "type": "list",
    "geometry": {"x": 0.1, "y": 0.2, "w": 0.8, "h": 0.6},
    "items": ["Point A", "Point B"],
    "align": "b"
}

Output 2:
\begin{textblock}{0.8}(0.1, 0.2)
  \begin{minipage}[b][0.6\paperheight]{\linewidth}
    \begin{itemize}
      \item Point A
      \item Point B
    \end{itemize}
  \end{minipage}
\end{textblock}

```

Die Stabilität ergab sich aus einer iterativen Testphase während der Entwicklung. Während Versuche ohne explizite Beispiele (siehe Listing 5) häufig zu variierenden LaTeX-Umgebungen und Syntaxfehlern führten, zeigte die Implementierung des In-Context-Musters eine konsistente Einhaltung des Zielschemas über den gesamten Testdatensatz hinweg. Diese Beobachtung deckt sich mit dem Forschungsstand zum In-Context Learning, wonach explizite Beispiele die Varianz des Modells bei strukturellen Aufgaben reduzieren [Brown et al., 2020].

3.5 Syntax-Validierung

Künstliche Intelligenz generiert Ergebnisse auf Basis von Wahrscheinlichkeiten. Dieses Verhalten führt systembedingt zu Halluzinationen, was auch bei der Erzeugung von LaTeX-Code auftritt. In diesem Abschnitt werden bekannte Fehler

abgefangen. Es erfolgt eine gezielte automatisierte Nachbereitung, um typische Flüchtigkeitsfehler in der Syntax zu korrigieren. Mittels regulärer Ausdrücke werden unvollständige Schlüsselwörter wie `\paper` erkannt und zu `\paperheight` ergänzt, um Kompilierungsfehler bei der Positionsbestimmung zu verhindern.

Die Implementierung der Steuerzeichen-Bereinigung folgt dem Ziel, die Kompilierstabilität durch deterministische Ersetzungsregeln zu sichern. Da Sprachmodelle aufgrund ihrer Tokenisierung dazu neigen, Backslashes fälschlicherweise als Schrägstriche oder Backspace-Zeichen auszugeben, transformiert die Routine diese Artefakte systematisch in valide LaTeX-Befehle (`begin`, `end`, `item`).

3.6 Kompilierung

Nach der sequenziellen Generierung sämtlicher Folienfragmente erfolgt die finale Aggregation des Dokuments. Hierbei wird der generierte Inhalt mit der initial erzeugten Präambel sowie der Postambel konkateniert, um ein valides `.tex`-Dokument zu bilden. Vor der Persistierung auf dem Dateisystem durchläuft der Quelltext einen Bereinigungsschritt, um potenzielle Inferenz-Artefakte des Sprachmodells zu entfernen.

Der Kompilierungsprozess wird über die subprocess-Schnittstelle gesteuert. Die Ausführung der `pdflatex`-Engine erfolgt innerhalb des Zielverzeichnis, wodurch die korrekte Auflösung relativer Pfade für eingebundene Medienressourcen sichergestellt wird. Zur Gewährleistung einer unterbrechungsfreien Pipeline wird der Compiler im Modus `-interaction=nonstopmode` betrieben. Diese Konfiguration steigert die Robustheit der automatisierten Verarbeitung, da der Satzvorgang bei geringfügigen Fehlern fortgesetzt wird und der Prozess lediglich bei kritischen Syntaxfehlern terminiert [Eijkhout [1991]].

- Validierung: Die erfolgreiche Erzeugung des PDF-Dokuments dient als primärer Indikator für die syntaktische Korrektheit des generierten Codes.
- Fehlerdiagnose: Im Falle eines Fehlschlags (Return-Code $\neq 0$) extrahiert das System die finalen Zeilen der Ursache. Dies ermöglicht eine gezielte Fehleranalyse.

4 Ergebnisse

Das vorliegende Kapitel evaluiert die Leistungsfähigkeit des entwickelten Systems. Im Fokus stehen dabei die Analyse der generierten Artefakte, die Identifikation technischer Limitationen sowie ein abschließender Vergleich mit existierenden Web-Tools.

4.1 Artefakte und Limitationen

Während der Transformation von PowerPoint-Folien in LaTeX-Code traten spezifische Problemstellungen auf, die die visuelle Treue des Ziel-Dokuments beeinflussen. Diese lassen sich in drei Kategorien unterteilen:

Designelemente: Ein Problem stellt die Extraktion von freien Formen und spezifischen PowerPoint-Shapes dar. Da diese innerhalb der Datenstruktur oft nur als generische Shapes ohne präzise Pfad-Spezifikationen referenziert werden, ist eine exakte Rekonstruktion in LaTeX aktuell nicht ohne Informationsverlust möglich. Ähnliches gilt für folienübergreifende Design-Templates und Hintergrundfarben: Da diese weder als Bilddateien noch als semantische Objekte extrahiert werden können, beschränkt sich der Output primär auf den funktionalen Content. Zu diesen nicht berücksichtigten Designelementen zählen insbesondere Farbschema, Folienthemen sowie spezifische Schriftformatierungen.

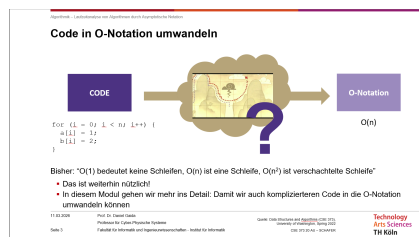


Abbildung 15: PowerPoint

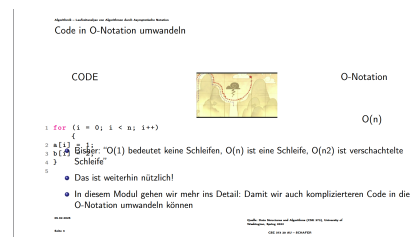


Abbildung 16: Latex

Datenintegrität bei der Medienextraktion: Die Evaluation der Medienextraktion via Docling erfolgte anhand eines Testdatensatzes von 8 Vorlesungsfolien aus der Algorithmik Vorlesung, die eine Mischung aus Text, mathematischen Formeln und eingebetteten Grafiken enthalten. Untersucht wurden dabei insbesondere die Medientypen Rastergrafiken (PNG, JPEG). Es zeigte sich, dass Docling grafische Inhalte nicht konsistent identifiziert, wenn diese semantisch mit Text vermischt sind.

Ein Beispiel liefert Abbildung 17: Die mathematischen Gleichungen (a–e) liegen im Quellformat nicht als Text, sondern als exportierte Grafikobjekte vor. Da Docling diese Objekte im Extraktionsprozess ignoriert, können sie im JSON nicht referenziert werden. Dies führt zu einem Inhaltsverlust im resultierenden Latex-Dokument, da Informationen der Folie fehlen (siehe Abbildung 20). Für eine vollständige Rekonstruktion ist hier eine alternative Extraktionsmethode notwendig.

4.1.1 Geometrische Inkonsistenzen bei Verschachtelungen

Ein technisches Problem bei der Verarbeitung Folienlayouts ist der Umgang mit Gruppierungen. Während einzeln stehende Elemente absolute Koordinaten relativ zum Folienrand besitzen, verhalten sich verschachtelte Elemente in der Datenstruktur anders. Die Extraktion via python-pptx erkennt zwar die Gruppierung, liefert für die darin enthaltenen Elemente jedoch Positionsdaten, die sich nicht auf die gesamte Folie beziehen.

Um dieses Verhalten zu untersuchen, wurde eine Folie mit verschachtelten Inhalten (s. Abbildung 17) analysiert. Dabei wurden die Rohdaten desselben Objekts in zwei Zuständen verglichen: einmal als Teil einer Gruppe und einmal nach dem manuellen Auflösen der Gruppierung.

Abbildung 17: Darstellung Slide 7 innerhalb des Powerpoints (Gruppirt)

O-, Ω- und Θ-Notation Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$? $\Theta(n^2)$?

- $f(n) = 42$
 $f(n) \in O(n^2)$
- $f(n) = 5n + 100$
 $f(n) \in O(n^2)$
- $f(n) = n \log_2(3n)$
 $f(n) \in O(n^2)$
- $f(n) = 4n^2 - 2n + 10$
 $f(n) \in O(n^2)$ $f(n) \in \Omega(n^2)$ $f(n) \in \Theta(n^2)$
- $f(n) = 2^n$
 $f(n) \in \Omega(n^2)$

O-Notation
 $f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ω-Notation
 $f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ-Notation
 $f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

05.02.2026 Prof. Dr. Daniel Gade
 Professor für Cyber-Physische Systeme
 Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373),
 University of Washington, Spring 2022
 CSE 373 20 SP - CHUN & CHAMPION

Technology
 Arts Sciences
 TH Köln

Der Vergleich der Extraktions-Logs in Listing 6 und Listing 7 offenbart die Ursache für die fehlerhafte Darstellung. In der gruppierten Struktur (Listing 6) wird das Bildelement Grafik 36 mit einem Left-Wert von ca. 3,35 cm ausgewiesen.

Listing 6: Auszug der Rohdaten: Gruppiertes Objekt

```
Shape: 'Gruppieren 34' (Type: GROUP (6))
Geo: Left: 7426683 (20.63cm), Top: 453359 (1.26cm) ...
[!] GROUP DETECTED - Checking Children:
Shape: 'Grafik 36' (Type: PICTURE (13))
Geo: Left: 1206500 (3.35cm), Top: 3618076 (10.05cm) ...
```

Im Gegensatz dazu zeigt Listing 7 dasselbe Bild (Grafik 36) nach der Auflösung der Gruppe. Hier wird die tatsächliche Position auf der Folie mit ca. 20,63 cm korrekt angegeben.

Listing 7: Auszug der Rohdaten: Entkoppeltes Objekt

```
Shape: 'Grafik 36' (Type: PICTURE (13))
Geo: Left: 7427146 (20.63cm), Top: 895227 (2.49cm) ...
```

Die Daten belegen, dass die Extraktions-Engine bei verschachtelten Objekten lediglich den lokalen Abstand zum inneren Rand der Gruppe misst, nicht jedoch die absolute Position auf der Folie.

Das Problem entsteht bei der Interpretation dieser Werte:

- Das Bild befindet sich visuell am rechten Rand der Folie (bei ca. 20,63 cm).
- Der Parser liefert jedoch den lokalen Wert von ca. 3,35 cm, da er den Startpunkt der Elterngruppe ignoriert.
- Da unser Algorithmus diesen Wert direkt als globale Koordinate interpretiert, wird das Bild im finalen LaTeX-Dokument fälschlicherweise 17 cm zu weit links platziert.

Abbildung 18: Layout-Kollaps durch Interpretation relativer Gruppen-Koordinaten als globale Werte.

Algorithmik – Laufzeitanalysen von Algorithmen durch Asymptotische Notation

O-, Ω- und Θ-Notation Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$? $\Theta(n^2)$?

a. $f(n) = O(n^2)$

b. $f(n) = \Omega(n^2)$

c. $f(n) = \Theta(n^2)$

e. $f(n) = \Omega(n^2)$

Ω-Notation

$f(n) \in \Theta(g(n))$ falls positive Konstanten c und n_0 existieren, sodass

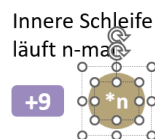
$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ für alle } n \geq n_0$$

08.02.2026
Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

Seite 7
CSE 373 20 SP – CHUN CHAMPION

Die Vermutung eines Parser-Fehlers greift hier zu kurz. Um zu verifizieren, dass dieses Verhalten deterministisch auftritt, wurden weitere Gruppierungen untersucht. Ein analoges Phänomen zeigt sich auf Folie 6 (siehe Abbildung 19). Auch hier entsprechen die ausgelesenen Koordinaten der Kind-Elemente exakt dem lokalen Abstand zum inneren Gruppenrand, was bei direkter Übernahme zu einer sichtbaren Verschiebung führt.

Abbildung 19: Gruppiertes Element (Folie 6).



Die Analyse der zugrunde liegenden OpenXML-Struktur offenbart die Ursache: Gruppierte Objekte spannen ein eigenes lokales Koordinatensystem auf. Der in Listing 6 ermittelte Wert von 3,35cm beschreibt die exakte Distanz zum internen Startpunkt der Gruppe. Das Element erbt den absoluten Startpunkt des Eltern-Elements nicht automatisch.

Abbildung 20: Valides Layout nach Auflösung der Gruppierung und Nutzung absoluter Koordinaten.

Algorithmen – Laufzeitanalyse von Algorithmen durch Asymptotische Notation
O-Notation

O-, Ω- und Θ-Notation Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$?

a. $f(n) \in O(n^2)$

b. $f(n) \in O(n^2)$

c. $f(n) \in O(n^2)$

d. $f(n) \in O(n^2)$

e. $f(n) \in \Omega(n^2)$

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Ω-Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ-Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

09.02.2026
Quelle: Data Structures and Algorithms (CSE 373) University of Washington, Spring 2012

Seite 7
CSE 373 20 SP – CHUN CHAMPHON

Zur Lösung dieses Referenzierungsproblems muss die Hierarchie der OpenXML-Datenstruktur aufgelöst werden. Das Format definiert für Gruppen zwei getrennte Koordinatensysteme: ein äußeres für die Position der Gruppe auf der Folie und ein inneres für die Anordnung der enthaltenen Elemente. Zwei Parameter aus der Dokumentation des Open XML SDK sind hierbei zentral:

- **chOff** (Child Offset): Der interne Startpunkt (Nullpunkt) des Gruppen-Koordinatensystems [Microsoft Corporation, 2024b].
- **chExt** (Child Extent): Die interne Ausdehnung (Breite/Höhe) der Gruppe [Microsoft Corporation, 2024a].

Kind-Elemente kennen ihre absolute Position auf der Folie nicht. Ihre Koordinaten beziehen sich ausschließlich auf dieses innere System. Die Unterscheidung ist notwendig, da Gruppen im Präsentationseditor skaliert werden können. Dabei ändert sich nur die äußere Breite der Gruppe (W_{Gruppe}), während die interne Ausdehnung (W_{chExt}) und die lokalen Positionen der Kind-Elemente starr bleiben.

Um die echte Folien-Koordinate (X_{abs}) eines Kind-Elements zu berechnen, wird die lokale Distanz zum internen Nullpunkt ermittelt, mit dem Skalierungsfaktor der Gruppe multipliziert und zur globalen Position der Elterngruppe addiert:

$$X_{abs} = X_{Gruppe} + (X_{Kind} - X_{chOff}) \cdot \frac{W_{Gruppe}}{W_{chExt}}$$

Die Funktionsweise dieser Transformation lässt sich anhand der Rohdaten aus Listing 6 für das Element *Grafik 36* nachvollziehen:

1. Startpunkt der Gruppe (X_{Gruppe}): Die Gruppe liegt bei 20,63 cm.
2. Lokale Position (X_{Kind}): Der ausgelesene Wert für das Bild liegt bei 3,35 cm.

3. Interner Nullpunkt (X_{chOff}): Die XML-Daten der Gruppe definieren den internen Startpunkt ebenfalls bei ca. 3,35 cm.
4. Skalierungsfaktor: In diesem Fall ist die Gruppe unskaliert, das Verhältnis von äußerer Breite zu chExt beträgt 1.

Eingesetzt in die Formel ergibt sich:

$$X_{abs} = 20,63 \text{ cm} + (3,35 \text{ cm} - 3,35 \text{ cm}) \cdot 1 = 20,63 \text{ cm}$$

Die Berechnung belegt, dass der lokale Wert von 3,35 cm lediglich beschreibt, dass das Bild direkt am linken inneren Rand der Gruppe anliegt. Durch die Transformation wird die absolute Folienposition von 20,63 cm rechnerisch ermittelt. Dies entspricht exakt dem Wert, den die API nach einer manuellen Auflösung der Gruppierung ausgibt (vgl. Listing 7).

Die Übertragung auf die y-Achse erfolgt nach demselben Muster, nutzt jedoch die vertikalen Parameter (Y_{Gruppe} , Y_{chOff}) sowie das Verhältnis von externer zu interner Gruppenhöhe (H_{Gruppe}/H_{chExt}).

Für dasselbe Element liest die python-pptx-Bibliothek einen lokalen Top-Wert (Y_{Kind}) von 10,05 cm aus. Die XML-Daten der Elterngruppe liefern die für die Transformation notwendigen restlichen Variablen: Die Gruppe selbst liegt bei 1,26 cm (Y_{Gruppe}), besitzt einen internen vertikalen Nullpunkt bei 8,73 cm (Y_{chOff}) und wird leicht herunterskaliert (externe Höhe 4,39 cm zu interner Höhe 4,73 cm).

Eingesetzt in die Formel ergibt sich für die absolute vertikale Position:

$$Y_{abs} = 1,26 \text{ cm} + (10,05 \text{ cm} - 8,73 \text{ cm}) \cdot \frac{4,39 \text{ cm}}{4,73 \text{ cm}} \approx 2,49 \text{ cm}$$

Auch dieses rechnerische Ergebnis deckt sich exakt mit den Daten, die nach der manuellen Auflösung der Gruppierung ausgegeben werden. Damit ist belegt, dass die Formel die Koordinaten deterministisch für alle Achsen auflöst.

4.2 Vergleich mit bestehenden Lösungen

Zur Einordnung der Ergebnisse wird das entwickelte System mit dem kommerziellen Konverter von Aspose verglichen [Aspose Pty Ltd, 2026]. Da der Hersteller keine technischen Details über den internen Konvertierungsalgorithmus veröffentlicht, basiert der folgende Vergleich auf einer empirischen Analyse des generierten Quellcodes.

Dabei zeigt sich, dass Aspose einen rein visuellen Export-Ansatz verfolgt. Anstatt semantische Strukturen wie Listen oder Textblöcke zu rekonstruieren, wird die Folie als grafische Komposition behandelt. Der resultierende LaTeX-Code nutzt die picture-Umgebung, in der mittels absoluter `\put`-Befehle jedes Textfragment oder Zeichen individuell positioniert wird (siehe Abbildung 21 und 23).

Abbildung 21: Latex-Code in Aspose Dokument

```

\begin{picture}(-5,0)(2.5,0)
\put(385.791,-241.49){\fontsize{20.013}{1}\usefont{T1}{ptm}{b}{n}\selectfont\color{color_283006}+}
\put(387.293,-265.499){\fontsize{20.013}{1}\usefont{T1}{ptm}{b}{n}\selectfont\color{color_283006}1}
\end{picture}
\begin{tikzpicture}[overlay]
\path(0pt,0pt);
\filldraw[color_183214][even odd rule]
(362.036pt,-359.468pt) -- (362.036pt,-359.468pt)
-- (362.036pt,-359.468pt) .. controls (362.036pt,-358.759pt) and (362.235pt,-358.079pt) .. (362.575pt,-357.483pt)
-- (362.575pt,-357.483pt) .. controls (362.915pt,-356.86pt) and (363.425pt,-356.35pt) .. (364.02pt,-356.009pt)
-- (364.02pt,-356.009pt) .. controls (364.644pt,-355.669pt) and (365.324pt,-355.471pt) .. (366.033pt,-355.471pt)
-- (366.033pt,-355.471pt)
-- (398.49pt,-355.499pt)
-- (398.49pt,-355.499pt)
-- (398.49pt,-355.499pt)
-- (398.49pt,-355.499pt) .. controls (399.198pt,-355.499pt) and (399.879pt,-355.698pt) .. (400.474pt,-356.038pt)
-- (400.474pt,-356.038pt) .. controls (401.098pt,-356.378pt) and (401.608pt,-356.888pt) .. (401.948pt,-357.483pt)
-- (401.948pt,-357.483pt) .. controls (402.288pt,-358.107pt) and (402.487pt,-358.787pt) .. (402.487pt,-359.496pt)
-- (402.487pt,-359.496pt)
-- (402.487pt,-359.496pt)
-- (402.487pt,-359.496pt)
-- (402.487pt,-375.455pt)
-- (402.487pt,-375.455pt)
-- (402.487pt,-375.455pt)
-- (402.487pt,-375.455pt) .. controls (402.487pt,-376.164pt) and (402.288pt,-376.844pt) .. (401.948pt,-377.439pt)
-- (401.948pt,-377.439pt) .. controls (401.608pt,-378.063pt) and (401.098pt,-378.573pt) .. (400.502pt,-378.913pt)
-- (400.502pt,-378.913pt) .. controls (399.879pt,-379.254pt) and (399.198pt,-379.452pt) .. (398.49pt,-379.452pt)
-- (398.49pt,-379.452pt)
-- (366.005pt,-379.452pt)
-- (366.005pt,-379.452pt)
-- (366.005pt,-379.452pt)
-- (366.005pt,-379.452pt) .. controls (365.296pt,-379.452pt) and (364.616pt,-379.254pt) .. (364.02pt,-378.913pt)
-- (364.02pt,-378.913pt) .. controls (363.397pt,-378.573pt) and (362.887pt,-378.063pt) .. (362.546pt,-377.468pt)
-- (362.546pt,-377.468pt) .. controls (362.206pt,-376.844pt) and (362.008pt,-376.164pt) .. (362.008pt,-375.455pt)
-- (362.008pt,-375.455pt)
-- (362.036pt,-359.468pt) -- cycle
;
\end{tikzpicture}
\begin{picture}(-5,0)(2.5,0)
\put(373.913,-363.493){\fontsize{20.013}{1}\usefont{T1}{ptm}{b}{n}\selectfont\color{color_283006}+}
\put(375.302,-387.502){\fontsize{20.013}{1}\usefont{T1}{ptm}{b}{n}\selectfont\color{color_283006}2}
\end{picture}

```

Diese Herweise bietet zwar eine hohe visuelle Treue zum Original, führt jedoch zu einem Verlust der semantischen Editierbarkeit:

- Ein großer Unterschied zeigt sich in der Quantität des Codes. Während der Aspose-Konverter für die vorliegende Test-Präsentation über 72.000 Zeilen generiert, beschränkt sich das entwickelte System auf lediglich ca. 800 Zeilen.
- Da kein zusammenhängender Text, sondern Fragmente auf Koordinaten generiert werden, ist der Code für eine manuelle Nachbearbeitung unbrauchbar. Es existieren keine logischen LaTeX-Umgebungen.
- Zwar werden Bilder im Code referenziert, findet keine Extraktion der Bilddateien aus dem .pptx-Container statt. Da der Konverter lediglich ein isoliertes .tex-Dokument ohne begleitende Ordner zurückgibt, fehlen die physischen Bilddaten bei der Kompilierung. Dies führt dazu, dass grafische Inhalte im finalen PDF nicht dargestellt werden können.
- Trotz der exakten Koordinaten kommt es häufig zu Verschiebungen bei Schriftarten und Textabständen, was das Schriftbild unruhig wirken lässt.

Im direkten Vergleich zwischen dem Resultat des Referenz-Tools (Abbildung 23) und dem hier entwickelten System (Abbildung 24) wird der konzeptionelle Gegensatz beider Ansätze deutlich. Während das Vergleichstool eine pixelgenaue, aber nicht editierbare Repräsentation erzielt, fokussiert sich das entwickelte System auf die Wiederherstellung logischer Strukturen. Obwohl der LLM-basierte Ansatz die ursprüngliche Designstruktur nicht identisch reproduziert, bleibt die semantische Informationsdichte, insbesondere bei Texten und Bildern, erhalten. Das Resultat ist ein wissenschaftlich nutzbares LaTeX-Dokument, das im

Gegensatz zur rein visuellen Kopie eine einfache manuelle Weiterverarbeitung ermöglicht.

Abbildung 22: Darstellung Slide 6 innerhalb des Powerpoints

Algorithmen – Laufzeitanalyse von Algorithmen durch Asymptotische Notation

Code Modellierung

Beispiel 2

```

public int method2(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        int j = 0;
        while (j < n) {
            if (j % 2 == 0) {
                // do nothing
            }
            sum = sum + (i * 3) + j;
            j = j + 1;
        }
        i = i + 1;
    }
    return sum;
}

```

$f(n) = (9n+4)n + 3$

Innere Schleife läuft n-mal
Äußere Schleife läuft n-mal

10.02.2026 Prof. Dr. Daniel Gade
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2002
CSE 373 20 AU - SCHAFER

Technology
Arts Sciences
TH Köln

Abbildung 23: Resultat des Aspose-Konverters

Algorithmen – Laufzeitanalyse von Algorithmen durch Asymptotische Notation

CodeModellierung

Beispiel 2

```

public int method2(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        int j = 0;
        while (j < n) {
            if (j % 2 == 0) {
                // do nothing
            }
            sum = sum + (i * 3) + j;
            j = j + 1;
        }
        i = i + 1;
    }
    return sum;
}

```

$f(n) = (9n+4)n + 3$

Innere Schleife läuft n-mal
Äußere Schleife läuft n-mal

10.02.2026 Prof. Dr. Daniel Gade
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2002
CSE 373 20 AU - SCHAFER

Technology
Arts Sciences
TH Köln

Abbildung 24: Resultat des eigenen Systems

Algorithmen – Laufzeitanalyse von Algorithmen durch Asymptotische Notation

Code Modellierung Beispiel 2

$f(n) = (9n+4)n + 3$

```

1 public int method2(int n) {
2   int sum = 0;
3   int i = 0;
4   while (i < n) {
5     int j = 0;
6     while (j < n) {
7       if (j % 2 == 0) {
8         // do nothing
9       }
10      sum = sum + (i * 3) + j;
11      j = j + 1;
12    }
13    i = i + 1;
14  } return sum;
15 }
16

```

Innere Schleife läuft n-mal
Äußere Schleife läuft n-mal

10.01.2026 Prof. Dr. Daniel Gade
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2002
CSE 373 20 AU - SCHAFER

5 Fazit

5.1 Zusammenfassung der Erkenntnisse

Ziel dieser Arbeit war die Entwicklung einer automatisierten Pipeline zur Konvertierung von PowerPoint-Präsentationen in LaTeX-Beamer-Code. Der implementierte Ansatz, der eine hybride Architektur aus klassischer Datenextraktion und generativen Sprachmodellen verfolgt, konnte die Machbarkeit dieses Vorhabens grundsätzlich bestätigen, zeigte jedoch klare Grenzen der aktuellen Technologien.

5.1.1 Reflexion des KI-Einsatzes

Trotz der Begeisterung für den Einsatz künstlicher Intelligenz in der Softwareentwicklung zeigt dieses Projekt, dass Large Language Models keine Universalwerkzeuge für komplexe, deterministische Probleme sind – insbesondere wenn es um die präzise Layout-Konstruktion in der UI geht.

Die größte Herausforderung bestand in der räumlichen Rekonstruktion. Sprachmodelle arbeiten probabilistisch – sie sagen das nächste Token basierend auf Wahrscheinlichkeiten voraus. Layout-Rendering erfordert jedoch absolute Präzision. Es zeigte sich, dass das Modell selbst bei präzisen Prompt-Anweisungen dazu neigt, Daten zu „halluzinieren“. Für Aufgaben, die feste Regeln erfordern, sind klassische, imperative Algorithmen daher durchaus noch eine Überlegung wert.

Abbildung 25: Beispiel für Modell-Halluzination.

Algorithmen - Laufzeitanalysen von Algorithmen durch Asymptotische Notation

O-Notation

O-, Ω- und Θ-Notation Übung

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$?

a. $f(n) = O(n^2)$

b. $f(n) = O(n^2)$

c. $f(n) = O(n^2)$

d. $f(n) = O(n^2)$

e. $f(n) = O(n^2)$

f. $f(n) = O(n^2)$

g. $f(n) = O(n^2)$

h. $f(n) = O(n^2)$

Handwritten notes:

- $f(n) \in O(g(n))$ falls positive Konstanten
- r und n , variationen endare
- $f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.

08.02.2026

Seite 7

Schon in Abbildung 25 lassen sich folgende Halluzinationen bemerken:

- Das Modell weist Textfragmenten falsche Schriftgrößen zu.
- Trotz konkreter Logik, werden Skalierungsparameter generiert.

Zusätzlich kann das Modell auch gestalterische Elemente wie Text erfinden, die im Originaldokument nicht existieren.

Ein paradoxer Effekt zeigte sich bei der Nutzung des Frameworks Docling. Obwohl dieses Tool explizit dafür konzipiert ist, Dokumente für die KI-Verarbeitung aufzubereiten, lieferte es für diesen spezifischen Anwendungsfall ein Übermaß an Informationen. Die generierten Datenstrukturen waren mit redundanten Daten überfrachtet, dass sie das Kontextfenster des Modells belasteten und eine aufwendige manuelle Bereinigung des JSON-Outputs erzwangen. (siehe Abschnitt 3.3)

5.2 Dezentrale Ansätze

Trotz der genannten Vorteile unterliegt die lokale Inferenz physischen Hardware-Limitierungen. Modelle mit hoher Parameteranzahl stoßen auf Standard-Systemen an Kapazitätsgrenzen, was die Verarbeitungszeit (siehe Abbildung 7) verlängert oder die Programmausführung aufgrund von Ressourcenengpässen gänzlich unterbindet. Einen Lösungsansatz bieten dezentrale Netzwerke wie *gonka.ai* [Liebermans, 2026].

Anstatt Rechenleistung exklusiv von einem einzelnen Unternehmen zu beziehen, werden hier verteilte Infrastrukturen erprobt. Plattformen wie *gonka.ai* basieren auf einem Protokoll, das ungenutzte GPU-Ressourcen einer Vielzahl unabhängiger Teilnehmer bündelt und koordiniert zur Verfügung stellt. Dies adressiert das Problem mangelnder lokaler Rechenkapazität, ohne die Souveränität über die Daten wieder vollständig an einen zentralen Großkonzern abzutreten. Solche dezentralen Architekturen könnten langfristig die Basis für unabhängige Open-Source-Tools bilden, da sie die Skalierbarkeit von Cloud-Lösungen mit dem Prinzip technologischer Autonomie verbinden.

Ein solcher Ansatz würde die Inferenz komplexer Modelle ermöglichen, die die Grenzen lokaler Standard-Hardware überschreiten.

5.3 Anwendungspotenzial

In der aktuellen Ausprägung wird die Pipeline primär als terminalbasierte Python-Applikation gesteuert. Während dies für die Validierung des technischen Konzepts ausreicht, stellt die fehlende grafische Benutzeroberfläche keine benutzerfreundliche Anwendung dar. Ein zentrales Ziel zukünftiger Versionen ist daher die Implementierung einer Steuerungsebene, die komplexe Konfigurationen – wie die Auswahl von Quell- und Zielverzeichnissen oder die Modellwahl – intuitiv zugänglich macht.

Ein weiterer Vorteil einer UI liegt in der gezielten Fehlerdiagnose. Anstatt systemnahe Logs auszugeben, könnten syntaktische Konflikte im LaTeX-Code visuell hervorgehoben und durch den Nutzer direkt korrigiert werden. Ergänzend dazu wäre ein User-Ansatz sinnvoll: Hierbei erhält der Nutzer die Möglichkeit, die berechneten Koordinaten der Zwischenrepräsentation vor der finalen Inferenz zu validieren. Dies würde es ermöglichen, die im Kapitel 4.1.1 identifizierten geometrischen Inkonsistenzen bei verschachtelten Gruppen manuell zu bereinigen, bevor die KI mit der Generierung des Zielcodes beginnt. Dies sichert die Layout-Treue auch in Fällen, in denen die automatisierte Extraktion an ihre strukturellen Grenzen stößt.

5.4 Abschließende Betrachtung

Die Arbeit zeigt, dass ein intelligenter Konverter mehr erfordert als nur ein leistungsfähiges Sprachmodell. Die Qualität des Outputs hängt maßgeblich von der Vorverarbeitung der Daten ab. Ein System, das die Stärken deterministischer Skripte mit der Flexibilität generativer KI vereint, stellt den vielversprechendsten Weg dar, um den manuellen Aufwand bei der Konvertierung komplexer Dokumente auf ein Minimum zu reduzieren.

Bei der Erstellung dieser Arbeit wurden KI-gestützte Werkzeuge zur Unterstützung bei der Strukturierung und zur sprachlichen Optimierung eingesetzt. Die inhaltliche Kontrolle und die finale Textfassung liegen vollständig beim Autor.

Literatur

- Jérôme Agater. Transpilierte sprachen, 2011. URL https://uol.de/f/2/dept/informatik/ag/parsys/agater_ma_folien.pdf.
- Unstructured API. Account api keys and api urls, 2026. URL <https://docs.unstructured.io/ui/account/api-key-url>.
- Aspose Pty Ltd. Aspose pptx to latex converter, 2026. URL <https://products.aspose.app/pdf/conversion/pptx-to-latex>. Online-Konverter.
- Christoph Auer, Maksym Lysak, Ahmed Nassar, et al. Docling technical report. Technical report, IBM Research, 2024. URL <https://arxiv.org/pdf/2408.09869>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- Steve Canny. *python-pptx documentation*, 2013. URL <https://python-pptx.readthedocs.io/en/latest/>.
- Unstructured Connectors, 2025. URL <https://unstructured.io/platform>. Beleg für No-Code-Schnittstellen und Konnektoren zu Cloud-Speichern.
- DML. Drawingml - overview, 2023. URL <http://officeopenxml.com/drw0overview.php>.
- DocDocs. Docling documentation: Supported formats, 2024. URL https://docling-project.github.io/docling/usage/supported_formats/. Abschnitt: Supported output formats.
- Docling. Docling - document parsing and analysis, 2026. URL <https://www.docling.ai/>.
- docling docs. Docling documentation, 2026. URL https://docling-project.github.io/docling/reference/docling_document/#docling_core.types.doc.DoclingDocument.
- Unstructured Docs. Supported file types - unstructured documentation, 2026. URL <https://docs.unstructured.io/open-source/introduction/supported-file-types>.
- Victor Eijkhout. *TeX by Topic: A TeXnician's Reference*. Addison-Wesley, Reading, Mass., 1991. URL <https://texdoc.org/serve/TeXbyTopic.pdf/0>. Chapter 32: nonstopmode.
- ISO/IEC. Information technology – document description and processing languages – office open xml file formats –. Technical report, International Organization for Standardization, 2016. URL <https://www.iso.org/obp/ui/en/#iso:std:iso-iec:29500:-1:ed-4:v1:en>. Kapitel 4 - Terms and Definitions - document category.

- Libermans. Gonka.ai - decentralized inference platform, 2026. URL <https://gonka.ai/>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Percy Liang, and Christopher D. Manning. Lost in the middle: How language models use long contexts. 2023. URL <https://arxiv.org/pdf/2307.03172>.
- LM Studio Team. Lm studio - local llm inference, 2026. URL <https://lmstudio.ai/>.
- Kim Martineau. Ibm research, 2026. URL <https://research.ibm.com/blog/docling-generative-AI>.
- Microsoft Corporation. Introducing the office (2007) open xml file formats, 2007. URL [https://learn.microsoft.com/de-de/previous-versions/office/developer/office-2007/aa338205\(v=office.12\)](https://learn.microsoft.com/de-de/previous-versions/office/developer/office-2007/aa338205(v=office.12)). Microsoft Developer Network.
- Microsoft Corporation. ChildExtents Class (DocumentFormat.OpenXml.Drawing), 2024a. URL <https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.drawing.childextents?view=openxml-3.0.1>.
- Microsoft Corporation. ChildOffset Class (DocumentFormat.OpenXml.Drawing), 2024b. URL <https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.drawing.childoffset?view=openxml-3.0.1>.
- olldeep. deepseek-r1:8b - ollama model library, 2025. URL <https://ollama.com/library/deepseek-r1:8b>.
- ollq25. Qwen2.5-coder:7b - ollama model library, 2025. URL <https://ollama.com/library/qwen2.5-coder:7b>.
- ollq3. Qwen3:8b - ollama model library, 2025. URL <https://ollama.com/library/qwen3:8b>.
- OTH Amberg-Weiden. Latex-portal der oth amberg-weiden hochschule, 2026. URL <https://www.oth-aw.de/hochschule/fakultaeten/elektrotechnik-medien-und-informatik/latex/>. Hochschul-Leitfaden zum wissenschaftlichen Arbeiten.
- Unstructured Pricing. Pricings, 2026. URL <https://unstructured.io/pricing>.
- Damai Dai Ce Zheng Jingyuan Ma Rui Li Heming Xia Jingjing Xu Zhiyong Wu Tianyu Liu Baobao Chang Xu Sun Lei Li Zhifang Sui Qingxiu Dong, Lei Li. A survey on in-context learning. 2022. URL <https://arxiv.org/abs/2301.00234>.
- qwen2.5-coder:7b. Testing benchmark, 2026. URL https://github.com/rippeddeadlift/CPS_pptx_latex_converter/tree/main/Results/TEST_4_SLIDES_qwen2.5-coder7b.

qwen3:8b. Testing benchmark, 2026. URL https://github.com/rippeddeadlift/CPS_pptx_latex_converter/tree/main/Results/TEST_4_SLIDES_qwen3_8b.

Deval Shah. What is in-context learning, and how does it work: The beginner's guide, 2026. URL <https://www.lakera.ai/blog/what-is-in-context-learning>.

The LaTeX Project. Latex – a document preparation system, 2026. URL <https://www.latex-project.org/>.

Unstructured Technologies. Unstructured - open source data processing for llms, 2026. URL <https://unstructured.io/>.

WordprocessingML. Xml-form eines wordprocessingml-dokuments, 2026. URL <https://learn.microsoft.com/de-de/dotnet/standard/linq/xml-shape-wordprocessingml-documents>.

Yiheng Xu et al. Layoutlm: Pre-training of text and layout for document image understanding, 2020. URL <https://dl.acm.org/doi/epdf/10.1145/3394486.3403172>. Proceedings of the 26th ACM SIGKDD. Page 1992 - ABSTRACT.